

Complete and Extensive practice of Docker-Compose

Scenario Based Questions (Final Armageddon)

Question 2: Automatic Startup of Dependent Services

Real-Life Situation:

A backend application requires a database to start first. If the database container is not ready, the backend fails to start.

Task:

Design a Docker Compose configuration where backend starts only after the database container.

Example:

services:

 backend:

 depends_on:

- database

Concepts Covered:

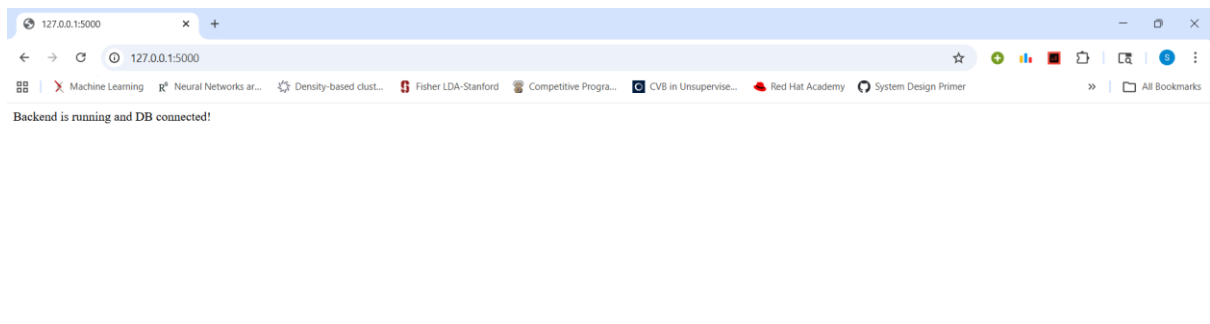
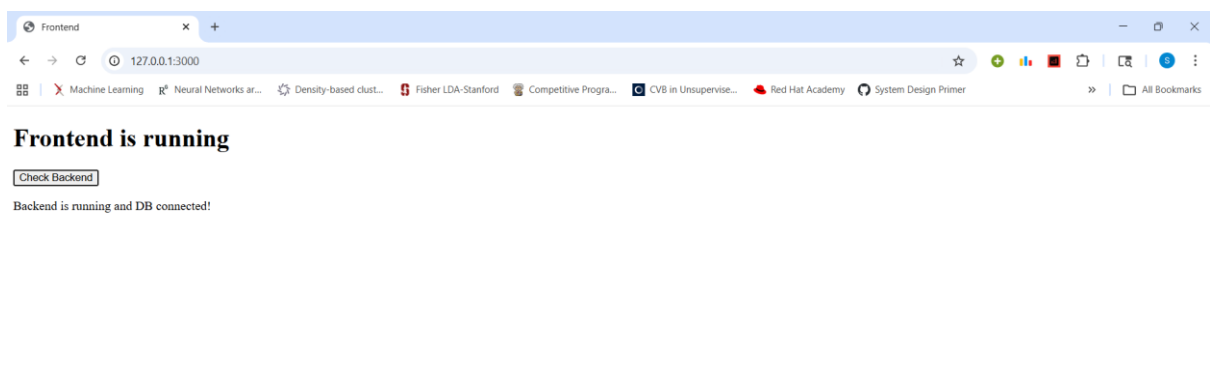
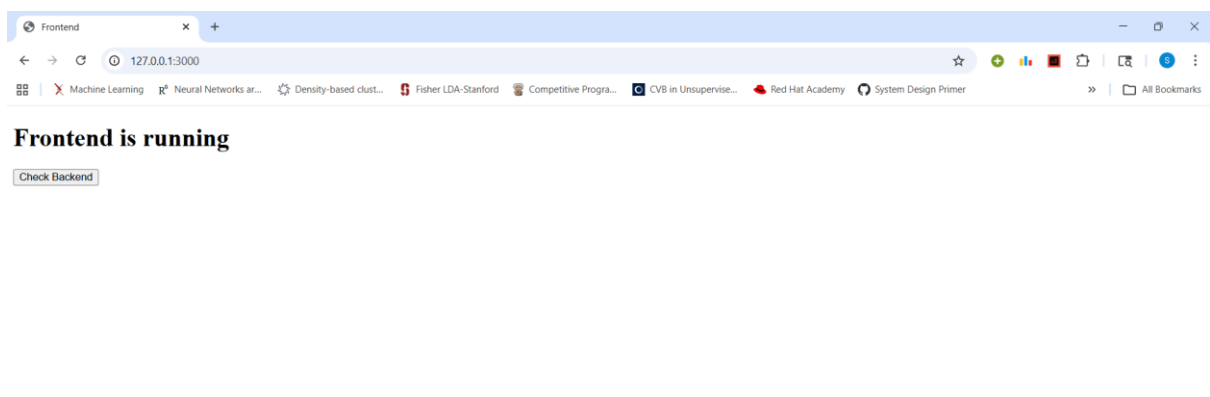
Service dependency ordering, Docker Compose orchestration

```
Terminal
#22 [backend] resolving provenance for metadata file
#22 DONE 0.1s
[+] up 4/4
✓ Image q2-frontend Built 25.3s
✓ Image q2-backend Built 25.3s
✓ Container backendCompose Recreated 0.7s
✓ Container frontendCompose Recreated 0.5s
Attaching to backendCompose, frontendCompose, mysql
mysql | 2026-04-24 09:14:34+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.8-1.el9 started.
Container mysql Waiting
mysql | 2026-04-24 09:14:35+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
mysql | 2026-04-24 09:14:35+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.8-1.el9 started.
mysql | '/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
mysql | 2026-04-24T09:14:35.668926Z 0 [System] [MY-015015] [Server] MySQL Server - start.
mysql | 2026-04-24T09:14:36.18883Z 1 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.4.8) starting as process 1
mysql | 2026-04-24T09:14:36.126790Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
mysql | 2026-04-24T09:14:36.822625Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
mysql | 2026-04-24T09:14:37.264285Z 0 [Warning] [MY-010668] [Server] CA certificate ca.pem is self signed.
mysql | 2026-04-24T09:14:37.264444Z 0 [System] [MY-013682] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for this channel.
mysql | 2026-04-24T09:14:37.278861Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different directory.
mysql | 2026-04-24T09:14:37.345535Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port: 33060, socket: /var/run/mysqld/mysqld.sock
mysql | 2026-04-24T09:14:37.345760Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.4.8' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community Server - GPL.
Container mysql Healthy
backendCompose | Connected to DB
backendCompose | * Serving Flask app 'app'
backendCompose | * Debug mode: off
backendCompose | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
backendCompose | * Running on all addresses (0.0.0.0)

RAM 2.40 GB CPU 2.66% Disk 57.98 GB used (limit 1006.85 GB) Terminal Update available
```

```
Terminal

frontendCompose | /docker-entrypoint.sh: Configuration complete; ready for start up
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: using the "epoll" event method
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: nginx/1.29.8
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: start worker processes
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: start worker process 30
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: start worker process 31
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: start worker process 32
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: start worker process 33
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: start worker process 34
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: start worker process 35
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: start worker process 36
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: start worker process 37
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: start worker process 38
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: start worker process 39
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: start worker process 40
frontendCompose | 2026/04/24 09:14:41 [notice] 1#1: start worker process 41
backendCompose | 192.168.16.1 - - [24/Apr/2026 09:14:52] "GET / HTTP/1.1" 200 -
frontendCompose | 192.168.16.1 - - [24/Apr/2026 09:15:05 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Ge
cko) Chrome/147.0.0.0 Safari/537.36" "-"
backendCompose | 192.168.16.1 - - [24/Apr/2026 09:15:07] "GET / HTTP/1.1" 200 -
```



```
services:
  database:
    image: mysql:8
    container_name: mysql
    restart: always
    env_file:
      - .env
    volumes:
      - db_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-p${MYSQL_ROOT_PASSWORD}"]
      interval: 5s
      timeout: 3s
      retries: 5

  backend:
    build: ./backend
    container_name: backendCompose
    restart: always
    env_file:
      - .env
    depends_on:
      database:
        condition: service_healthy
    ports:
      - "5000:5000"

  frontend:
    build: ./frontend
    container_name: frontendCompose
    depends_on:
      - backend
    ports:
      - "3000:80"

volumes:
  db_data:
```

Ln 30, Col 36 698 characters Plain text 100% Windows (CRLF) UTF-8

```
from flask import Flask
from flask_cors import CORS
import mysql.connector
import os
import time

app = Flask(__name__)
CORS(app)

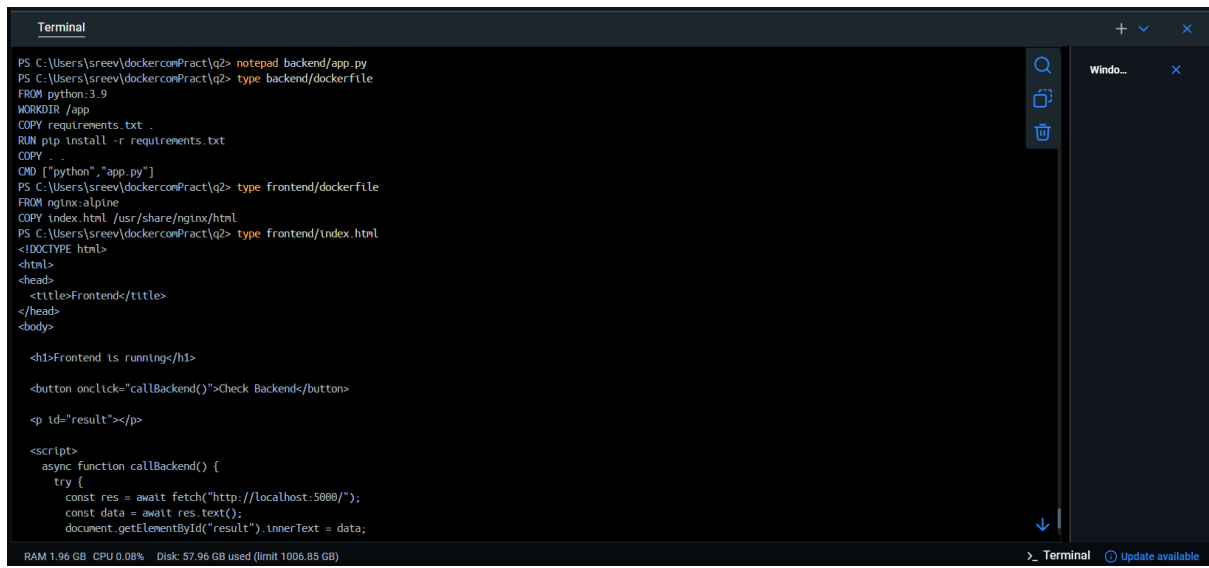
def connect():
    while True:
        try:
            conn = mysql.connector.connect(
                host=os.getenv("DB_HOST"),
                user=os.getenv("DB_USER"),
                password=os.getenv("DB_PASSWORD"),
                database=os.getenv("DB_NAME")
            )
            print("Connected to DB")
            return conn
        except:
            print("Waiting for DB...")
            time.sleep(2)

conn = connect()

@app.route("/")
def home():
    return "Backend is running and DB connected!"

app.run(host="0.0.0.0", port=5000)
```

Ln 9, Col 1 690 characters Plain text 100% Windows (CRLF) UTF-8



```
Terminal
PS C:\Users\sreev\dockercomPract\q2> notepad backend/app.py
PS C:\Users\sreev\dockercomPract\q2> type backend/dockerfile
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
PS C:\Users\sreev\dockercomPract\q2> type frontend/dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html
PS C:\Users\sreev\dockercomPract\q2> type frontend/index.html
<!DOCTYPE html>
<html>
<head>
<title>frontend</title>
</head>
<body>

<h1>Frontend is running</h1>

<button onclick="callBackend()">Check Backend</button>

<p id="result"></p>

<script>
  async function callBackend() {
    try {
      const res = await fetch("http://localhost:5000/");
      const data = await res.text();
      document.getElementById("result").innerText = data;
    } catch (error) {
      console.log(error);
    }
  }
</script>
</body>
</html>
```

RAM 1.96 GB CPU 0.08% Disk: 57.96 GB used (limit 1006.85 GB) >_ Terminal Update available

Question 3: Environment Configuration for Microservices

Real-Life Situation:

A Node.js backend service must connect to MongoDB using environment variables.

Task:

Create a Docker Compose configuration using environment variables to configure the database connection.

Example:

services:

 backend:

 image: node

 environment:

 - DB_HOST=mongodb

Concepts Covered:

Environment variables, Microservice configuration


```
Terminal
974", "isLoadBalanced": false, "uid": {"$uid": {"$uid": "84c1fa1f-a1f1-470f-a8e3-f217d9fcb3dc"}}, "connectionId": 4, "connectionCount": 2}}
mongodb | [{"t":{"date":"2026-04-26T17:43:07.994+00:00"},"s":"I", "c":"NETWORK", "id":22944, "ctx":"conn3","msg":"Connection ended","attr":{"remote":"127.0.0.1:35
964", "isLoadBalanced": false, "uid": {"$uid": {"$uid": "85541978-f075-476c-aa23-5bd25869ad9"}}, "connectionId": 3, "connectionCount": 1}}
mongodb | [{"t":{"date":"2026-04-26T17:43:07.994+00:00"},"s":"I", "c":"NETWORK", "id":22944, "ctx":"conn2","msg":"Connection ended","attr":{"remote":"127.0.0.1:35
962", "isLoadBalanced": false, "uid": {"$uid": {"$uid": "0972d0b1-debc-4931-9801-c53caf73bde"}}, "connectionId": 2, "connectionCount": 0}}
Container mongodb Healthy

mongodb | [{"t":{"date":"2026-04-26T17:43:09.174+00:00"},"s":"I", "c":"NETWORK", "id":22943, "ctx":"listener","msg":"Connection accepted","attr":{"remote":"192.16
8.32.4:44084", "isLoadBalanced": false, "uid": {"$uid": {"$uid": "bde435-2a7e-490a-909f-zb5cd5093588"}}, "connectionId": 6, "connectionCount": 1}}

RAM 4.30 GB CPU 8.98% Disk: 59.61 GB used (limit 1006.85 GB) >_ Terminal Update available
```

```
Terminal
services:
mongodb:
  image: mongo:7
  container_name: mongodb
  restart: unless-stopped
  env_file:
    - mongodb
  condition: service_healthy

frontend:
  build: ./frontend
  container_name: frontend3
  restart: unless-stopped
  ports:
    - "3000:80"

volumes:
  mongo_data:
PS C:\Users\sreev\docker-comp3> type .env
MONGO_INITDB_ROOT_USERNAME=admin
MONGO_INITDB_ROOT_PASSWORD=root
MONGO_DB=my_db

DB_HOST=mongodb
DB_PORT=27017
DB_USER=admin
DB_PASSWORD=root
DB_NAME=my_db

PORT=5000
NODE_ENV=production

RAM 4.02 GB CPU 0.17% Disk: 59.61 GB used (limit 1006.85 GB) >_ Terminal Update available
```

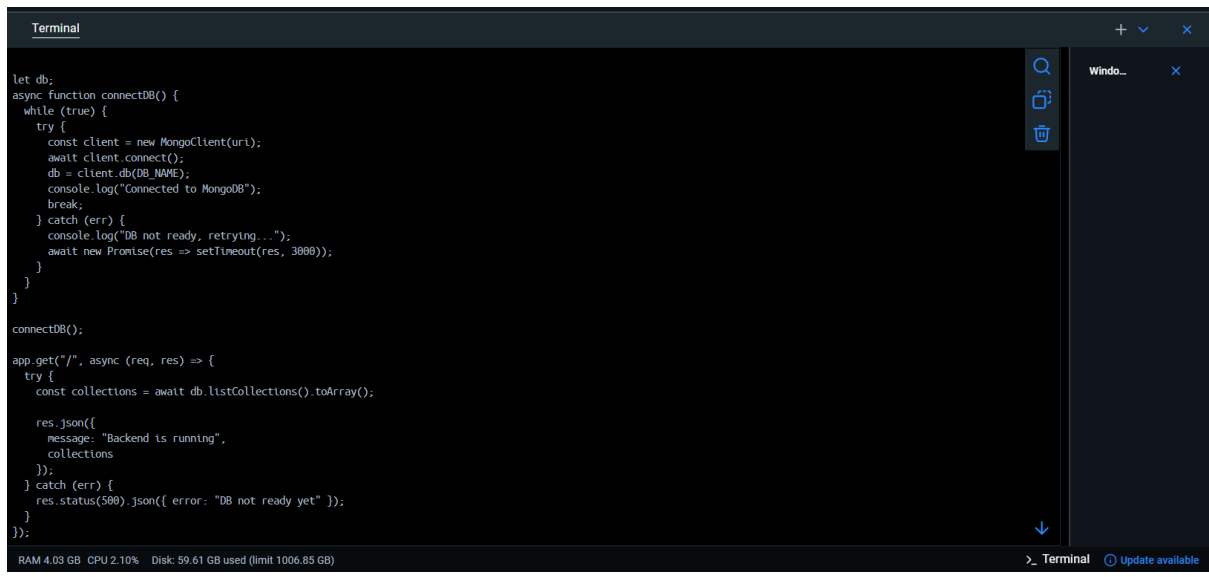
```
Terminal
server{
  listen 80;
  location / {
    root /usr/share/nginx/html;
    index index.html;
  }
  location /api/ {
    proxy_pass http://backend:5000/;

    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
  }
}
PS C:\Users\sreev\docker-comp3> type backend/server.js
const express = require('express');
const { MongoClient } = require('mongodb');

const app = express();

const [
  DB_HOST,
  DB_PORT,
  DB_USER,
  DB_PASSWORD,
  DB_NAME,
  PORT
] = process.env;

const url = `mongodb://${DB_USER}:${DB_PASSWORD}@${DB_HOST}:${DB_PORT}/${DB_NAME}?authSource=admin`;
```



```
Terminal
let db;
async function connectDB() {
  while (true) {
    try {
      const client = new MongoClient(uri);
      await client.connect();
      db = client.db(DB_NAME);
      console.log("Connected to MongoDB");
      break;
    } catch (err) {
      console.log("DB not ready, retrying...");
      await new Promise(res => setTimeout(res, 3000));
    }
  }
}

connectDB();

app.get("/", async (req, res) => {
  try {
    const collections = await db.listCollections().toArray();

    res.json({
      message: "Backend is running",
      collections
    });
  } catch (err) {
    res.status(500).json({ error: "DB not ready yet" });
  }
});
```

RAM 4.03 GB CPU 2.10% Disk: 59.61 GB used (limit 1006.85 GB)

Terminal Update available

Question 5: Multi-Container Deployment (WordPress + MySQL)

Real-Life Situation:

A blogging platform needs WordPress running with a MySQL database.

Task:

Deploy WordPress and MySQL services using Docker Compose.

Example:

services:

wordpress:

image: wordpress

mysql:

image: mysql

Run using:

docker-compose up -d

Concepts Covered:

Multi-container applications, Docker Compose orchestration

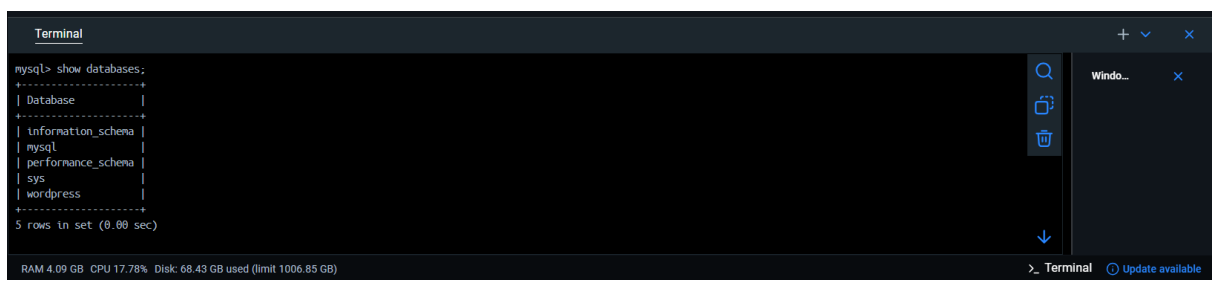
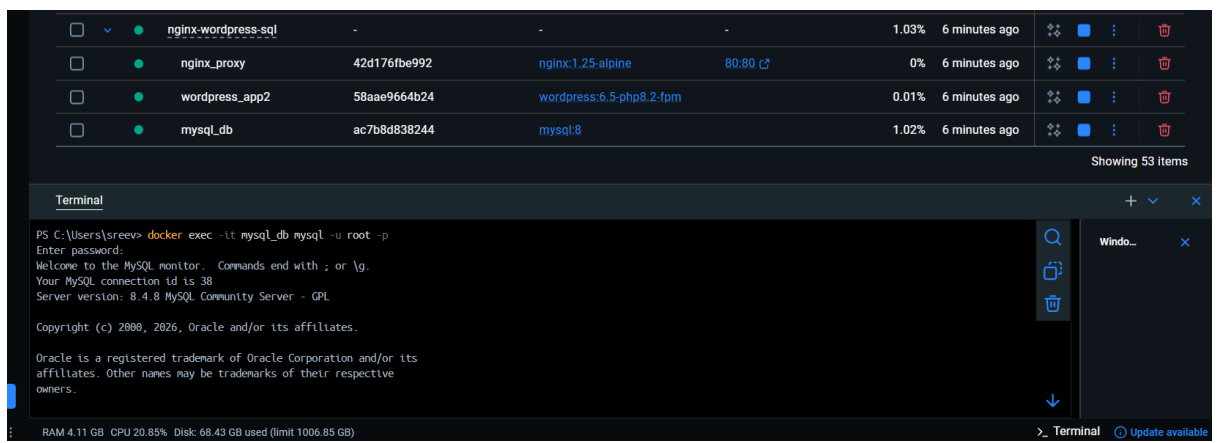
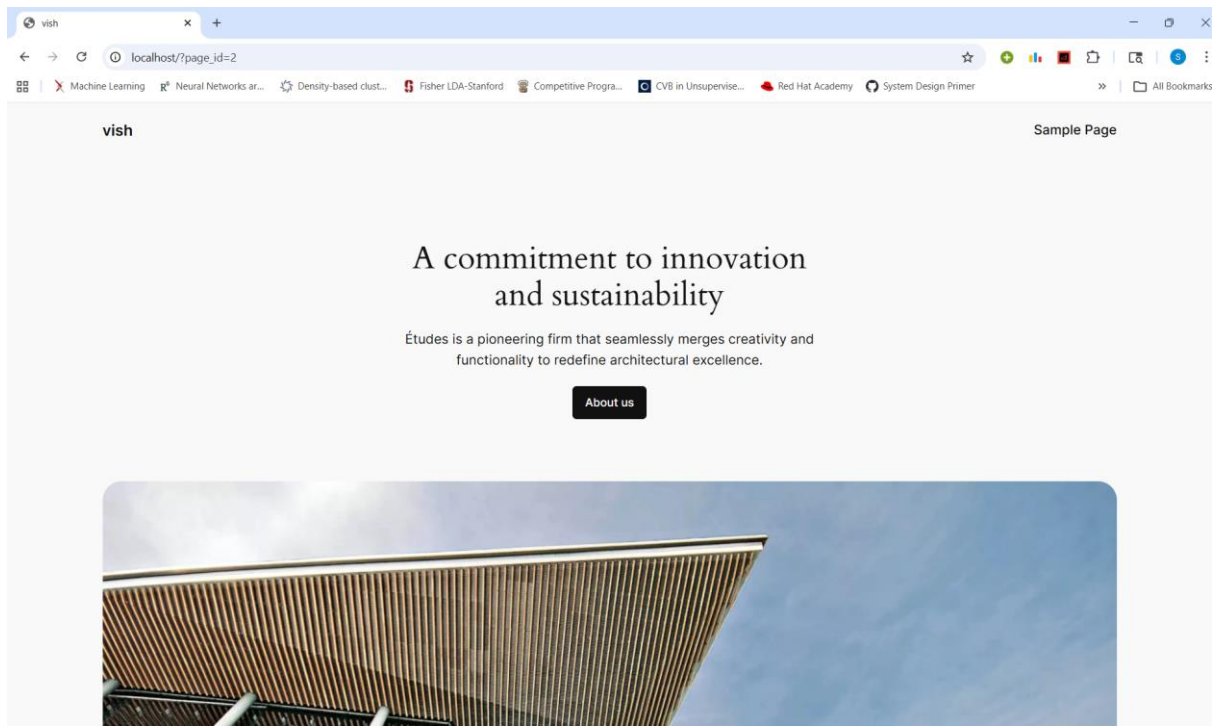
```
Command Prompt - docker c X + v

C:\Users\sreev\nginx-wordpress-sql>docker compose up --build
[+] up 7/7
✔Network nginx-wordpress-sql_backend      Created      0.0s
✔Network nginx-wordpress-sql_frontend     Created      0.1s
✔Volume nginx-wordpress-sql_mysql_data    Created      0.0s
✔Volume nginx-wordpress-sql_wordpress_data Created      0.0s
✔Container mysql_db                       Created      0.2s
✔Container wordpress_app2                 Created      0.3s
✔Container nginx_proxy                    Created      0.2s
Attaching to mysql_db, nginx_proxy, wordpress_app2
mysql_db | 2026-05-04 09:20:00+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.8-1.el9 started.
Container mysql_db Waiting
mysql_db | 2026-05-04 09:20:00+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
mysql_db | 2026-05-04 09:20:00+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.8-1.el9 started.
mysql_db | 2026-05-04 09:20:00+00:00 [Note] [Entrypoint]: Initializing database files
mysql_db | 2026-05-04T09:20:00.727970Z 0 [System] [MY-015017] [Server] MySQL Server Initialization - start.
mysql_db | 2026-05-04T09:20:00.729391Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.4.8) initializing of
server in progress as process 80
mysql_db | 2026-05-04T09:20:00.737658Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
mysql_db | 2026-05-04T09:20:01.175294Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
mysql_db | 2026-05-04T09:20:03.155820Z 6 [Warning] [MY-010453] [Server] root@localhost is created with an empty passwor
d ! Please consider switching off the --initialize-insecure option.
mysql_db | 2026-05-04T09:20:06.346705Z 0 [System] [MY-015018] [Server] MySQL Server Initialization - end.
mysql_db | 2026-05-04 09:20:06+00:00 [Note] [Entrypoint]: Database files initialized
mysql_db | 2026-05-04 09:20:06+00:00 [Note] [Entrypoint]: Starting temporary server
mysql_db | 2026-05-04T09:20:06.447692Z 0 [System] [MY-015015] [Server] MySQL Server - start.
mysql_db | 2026-05-04T09:20:06.728800Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.4.8) starting as proc
ess 129
mysql_db | 2026-05-04T09:20:06.747645Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
```

```
Command Prompt - docker c X + v

nginx_proxy | 192.168.80.1 - - [04/May/2026:09:21:20 +0000] "GET /wp-content/themes/twentytwentyfour/assets/images/w
indows.webp HTTP/1.1" 200 126244 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML
L, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
nginx_proxy | 192.168.80.1 - - [04/May/2026:09:21:20 +0000] "GET /wp-content/themes/twentytwentyfour/assets/fonts/in
ter/Inter-VariableFont_slnt,wght.woff2 HTTP/1.1" 200 326628 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x6
4) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
nginx_proxy | 192.168.80.1 - - [04/May/2026:09:21:20 +0000] "GET /wp-content/themes/twentytwentyfour/assets/fonts/ca
rdo/cardo_normal_400.woff2 HTTP/1.1" 200 146060 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebK
it/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
nginx_proxy | 192.168.80.1 - - [04/May/2026:09:21:20 +0000] "GET /wp-content/themes/twentytwentyfour/assets/fonts/ca
rdo/cardo_italic_400.woff2 HTTP/1.1" 200 105184 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebK
it/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
nginx_proxy | 192.168.80.1 - - [04/May/2026:09:21:20 +0000] "GET /wp-includes/js/wp-emoji-release.min.js?ver=6.5.5 H
TTP/1.1" 200 18726 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/147.0.0.0 Safari/537.36" "-"
wordpress_app2 | 192.168.80.3 - 04/May/2026:09:21:20 +0000 "GET /index.php" 200
nginx_proxy | 192.168.80.1 - - [04/May/2026:09:21:20 +0000] "GET /favicon.ico/ HTTP/1.1" 200 84712 "http://localhost
/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"

wordpress_app2 | 192.168.80.3 - 04/May/2026:09:21:27 +0000 "GET /index.php" 200
nginx_proxy | 192.168.80.1 - - [04/May/2026:09:21:27 +0000] "GET /?page_id=2 HTTP/1.1" 200 84712 "http://localhost/"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
wordpress_app2 | 192.168.80.3 - 04/May/2026:09:21:27 +0000 "GET /index.php" 200
nginx_proxy | 192.168.80.1 - - [04/May/2026:09:21:27 +0000] "GET /favicon.ico/ HTTP/1.1" 200 84712 "http://localhost
/?page_id=2" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/5
37.36" "-"
```



```
Command Prompt
services:
  mysql:
    image: mysql:8
    container_name: mysql_db
    restart: unless-stopped
    env_file:
      - .env
    networks:
      - backend
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -u root -p${MYSQL_ROOT_PASSWORD}"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 20s

  wordpress:
    image: wordpress:6.5-php8.2-fpm
    container_name: wordpress_app2
    restart: unless-stopped
    env_file: .env
    environment:
      WORDPRESS_DB_HOST: mysql:3306
      WORDPRESS_DB_USER: ${MYSQL_USER}
      WORDPRESS_DB_PASSWORD: ${MYSQL_PASSWORD}
      WORDPRESS_DB_NAME: ${MYSQL_DATABASE}
    networks:
      - backend
```

```
Command Prompt
  - frontend
  volumes:
    - wordpress_data:/var/www/html
  depends_on:
    mysql:
      condition: service_healthy

  nginx:
    image: nginx:1.25-alpine
    container_name: nginx_proxy
    restart: unless-stopped
    ports:
      - "80:80"
    volumes:
      - wordpress_data:/var/www/html:ro
      - ./nginx/conf.d:/etc/nginx/conf.d:ro
    networks:
      - frontend
    depends_on:
      - wordpress

networks:
  backend:
    internal: true
  frontend:

volumes:
  mysql_data:
  wordpress_data:
C:\Users\sreev\nginx-wordpress-sql>
```

A screenshot of a code editor window with multiple tabs. The active tab is 'defau'. The editor displays an Nginx configuration file with the following content:

```
server {  
    listen 80;  
  
    root /var/www/html;  
    index index.php;  
  
    location / {  
        try_files $uri /index.php;  
    }  
  
    location ~ \.php$ {  
        fastcgi_pass wordpress:9000;  
        include fastcgi_params;  
        fastcgi_param SCRIPT_FILENAME /var/www/html$fastcgi_script_name;  
    }  
}
```

conf.d file (reverse proxy)

Question 7: Deploying Java Spring Boot + PostgreSQL

Real-Life Situation:

A fintech startup runs a Spring Boot backend service that connects to PostgreSQL database.

Task:

Deploy Spring Boot and PostgreSQL containers using Docker Compose.

Example:

services:

springboot:

image: springboot-app

postgres:

image: postgres

Concepts Covered:

Multi-container deployment, Backend + database architecture

```
Command Prompt
Directory of D:\postgre-springboot-docker
12-05-2026 11:14 <DIR> .
12-05-2026 11:34 1,030 docker-compose.yml
12-05-2026 11:42 <DIR> spring-app
1 File(s) 1,030 bytes
2 Dir(s) 137,631,961,088 bytes free

D:\postgre-springboot-docker>cd spring-app

D:\postgre-springboot-docker\spring-app>dir
Volume in drive D is New Volume
Volume Serial Number is AEE6-4A19

Directory of D:\postgre-springboot-docker\spring-app
12-05-2026 11:42 <DIR> .
12-05-2026 11:14 <DIR> ..
12-05-2026 10:55 38 .gitattributes
12-05-2026 10:55 394 .gitignore
12-05-2026 10:55 <DIR> .mvn
12-05-2026 11:05 136 Dockerfile
12-05-2026 10:55 1,447 HELP.md
12-05-2026 10:55 11,790 mvnw
12-05-2026 10:55 8,292 mvnw.cmd
12-05-2026 10:55 1,479 pom.xml
12-05-2026 10:55 <DIR> src
12-05-2026 11:42 <DIR> target
7 File(s) 23,576 bytes
5 Dir(s) 137,631,961,088 bytes free
```

Machine Learning

Neural Networks ar...

Density-based clust...

Fisher LDA-Stanford

Competitive Progra...

CVB in Unsupervise...

Red Hat Academy

System Design Primer

All Bookmark...

spring initializr

Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Maven

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT)

☐ 4.1.0 (RC1)

☐ 4.0.7 (SNAPSHOT)

☐ 4.0.6

☐ 3.5.15 (SNAPSHOT)

☒ 3.5.14

Project Metadata

Group

com.example

Artifact

spring-app

Package name

com.example.spring-app

Packaging

☒ Jar

☐ War

Configuration

☒ Properties

☐ YAML

Java

☐ 26

☐ 25

☐ 21

☒ 17

Dependencies

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

PostgreSQL Driver

SQL

A JDBC and R2DBC driver that allows Java programs to connect to a PostgreSQL database using standard, database independent Java code.

ADD DEPENDENCIES... CTRL + B

GENERATE CTRL + G

EXPLORE CTRL + SPACE

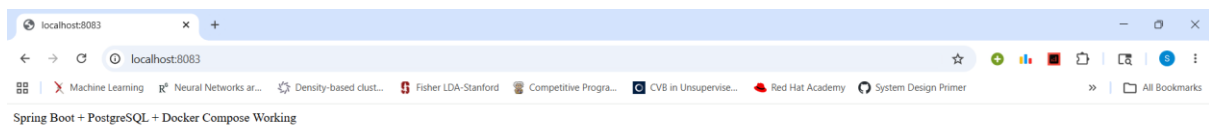
...

```
Command Prompt - mvnw.crr  +  v
2026-05-12T13:39:33.606+05:30 INFO 20508 --- [spring-app] [main] c.e.s.SpringAppApplicationTests :
No active profile set, falling back to 1 default profile: "default"
2026-05-12T13:39:35.217+05:30 INFO 20508 --- [spring-app] [main] c.e.s.SpringAppApplicationTests :
Started SpringAppApplicationTests in 2.01 seconds (process running for 3.287)
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.337 s -- in com.example.spring_app.SpringAppApplicationTests
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jar:3.4.2:jar (default-jar) @ spring-app ---
[INFO] Building jar: D:\postgre-springboot-docker\spring-app\target\spring-app-0.0.1-SNAPSHOT.jar
[INFO] --- spring-boot:3.5.14:repackage (repackage) @ spring-app ---
[INFO] Replacing main artifact D:\postgre-springboot-docker\spring-app\target\spring-app-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to D:\postgre-springboot-docker\spring-app\target\spring-app-0.0.1-SNAPSHOT.jar.original
[INFO] BUILD SUCCESS
[INFO] Total time: 8.794 s
[INFO] Finished at: 2026-05-12T13:39:37+05:30
[INFO]
D:\postgre-springboot-docker\spring-app>
```

```
Command Prompt - mvnw.crr  +  v
#9 exporting manifest sha256:e5b17b413e940fce26b251afcd5c7ccfba50f331a982cfc82bbc0a296e6856df 0.0s done
#9 exporting config sha256:79d72c0eca79a5a42736817a77926c727153834d9b5ab5488b98e068e6cc892c 0.0s done
#9 exporting attestation manifest sha256:baf4be3df80ebb0ad81c0a57a74872c48e26d60e0499a314743e2868b2630471 0.0s done
#9 exporting manifest list sha256:f928a7f41979deacd9bd1a6e13637e5552e1be189cc51c4c2c1ca5cc30b7e6bd 0.0s done
#9 exporting manifest list sha256:f928a7f41979deacd9bd1a6e13637e5552e1be189cc51c4c2c1ca5cc30b7e6bd 0.0s done
#9 naming to docker.io/library/postgre-springboot-docker-springboot-app:latest done
#9 unpacking to docker.io/library/postgre-springboot-docker-springboot-app:latest
#9 unpacking to docker.io/library/postgre-springboot-docker-springboot-app:latest 0.3s done
#9 DONE 1.3s

#10 resolving provenance for metadata file
#10 DONE 0.0s
[+] up 2/2
  ✔ Image postgre-springboot-docker-springboot-app Built 4.2s
  ✔ Container springboot-app-container Recreated 0.3s
Attaching to postgres-db, springboot-app-container
Container postgres-db Waiting
postgres-db | PostgreSQL Database directory appears to contain a database; Skipping initialization
postgres-db |
postgres-db | 2026-05-12 08:11:29.976 UTC [1] LOG: starting PostgreSQL 17.9 (Debian 17.9-1.pgdg13+1) on x86_64-pc-linu
x-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
postgres-db | 2026-05-12 08:11:29.977 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
postgres-db | 2026-05-12 08:11:29.977 UTC [1] LOG: listening on IPv6 address ":::", port 5432
postgres-db | 2026-05-12 08:11:29.985 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
postgres-db | 2026-05-12 08:11:29.995 UTC [27] LOG: database system was shut down at 2026-05-12 06:13:54 UTC
postgres-db | 2026-05-12 08:11:30.015 UTC [1] LOG: database system is ready to accept connections
```

```
Command Prompt - mvnw.cnr x + v
Container postgres-db Healthy
springboot-app-container
springboot-app-container
springboot-app-container
springboot-app-container
springboot-app-container
springboot-app-container
springboot-app-container
springboot-app-container
springboot-app-container
springboot-app-container
Application : Starting SpringAppApplication v0.0.1-SNAPSHOT using Java 17.0.18 with PID 1 (/app/app.jar started by
root in /app)
springboot-app-container | 2026-05-12T08:11:37.461Z INFO 1 --- [spring-app] [ main] c.e.spring_app.SpringApp
Application : No active profile set, falling back to 1 default profile: "default"
springboot-app-container | 2026-05-12T08:11:37.469Z INFO 1 --- [spring-app] [ main] c.e.spring_app.SpringApp
springboot-app-container | 2026-05-12T08:11:38.834Z INFO 1 --- [spring-app] [ main] o.s.b.w.embedded.tomcat.
TomcatWebServer : Tomcat initialized with port 8080 (http)
springboot-app-container | 2026-05-12T08:11:38.855Z INFO 1 --- [spring-app] [ main] o.apache.catalina.core.S
standardService : Starting service [Tomcat]
springboot-app-container | 2026-05-12T08:11:38.855Z INFO 1 --- [spring-app] [ main] o.apache.catalina.core.S
standardEngine : Starting Servlet engine: [Apache Tomcat/10.1.54]
springboot-app-container | 2026-05-12T08:11:38.893Z INFO 1 --- [spring-app] [ main] o.a.c.c.C.[Tomcat].[loca
lhost].[/] : Initializing Spring embedded WebApplicationContext
springboot-app-container | 2026-05-12T08:11:38.896Z INFO 1 --- [spring-app] [ main] w.s.c.ServletWebServerAp
plicationContext : Root WebApplicationContext: initialization completed in 1327 ms
springboot-app-container | 2026-05-12T08:11:39.516Z INFO 1 --- [spring-app] [ main] o.s.b.w.embedded.tomcat.
TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
springboot-app-container | 2026-05-12T08:11:39.548Z INFO 1 --- [spring-app] [ main] c.e.spring_app.SpringApp
Application : Started SpringAppApplication in 2.906 seconds (process running for 3.83)
```



```
Command Prompt - mvnw.cnr x + v
services:
  springboot-app:
    build:
      context: ./spring-app
      container_name: springboot-app-container
    ports:
      - "8083:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/my_db
      SPRING_DATASOURCE_USERNAME: myuser
      SPRING_DATASOURCE_PASSWORD: mypassword
    depends_on:
      postgres:
        condition: service_healthy
    restart: unless-stopped
    networks:
      - backend-network

  postgres:
    image: postgres:17
    container_name: postgres-db
    ports:
      - "5432:5432"
    environment:
      POSTGRES_DB: my_db
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
```

Question 8: Scaling Microservices for High Traffic

Real-Life Situation:

An online learning platform receives sudden high traffic during exams and needs to scale backend services.

Task:

Demonstrate how to scale backend containers using Docker Compose.

Example:

`docker-compose up --scale backend=3`

Concepts Covered:

Scalability, Microservices architecture

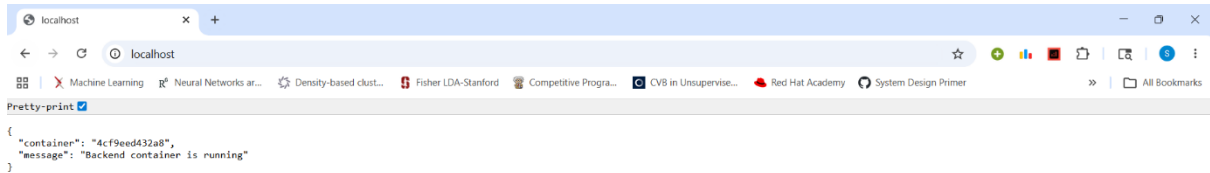
```
Command Prompt
Volume Serial Number is AEE6-4A19

Directory of D:\scaling-demo
13-05-2026 18:17 <DIR> .
13-05-2026 17:44 <DIR> backend
13-05-2026 18:35 530 docker-compose.yml
13-05-2026 17:48 <DIR> nginx
                  1 File(s) 530 bytes
                  3 Dir(s) 137,628,246,016 bytes free

D:\scaling-demo>type docker-compose.yml
services:
  backend:
    build: ./backend
    expose:
      - "8080"
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080"]
      interval: 10s
      timeout: 3s
      retries: 5
      start_period: 10s

  frontend:
    image: nginx:latest
    container_name: frontendcont
    ports:
      - "80:80"
    depends_on:
      backend:
        condition: service_healthy
    restart: unless-stopped
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
D:\scaling-demo>
```

```
D:\scaling-demo\backend>type dockerfile
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y curl
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
D:\scaling-demo\backend>
```



Before Scaling

```
Command Prompt

#12 exporting manifest sha256:341d000778c53071c6e991f275dc900872c320f5ed1b65a0dfcea8c7735985a7 done
#12 exporting config sha256:b5ca3c49a335b5762cabf274934f146f6f3f08ec918543aa1a77dc332f698c39 done
#12 exporting attestation manifest sha256:ef1f356ebe3e371e61c13b30579095e127ac6130264c02016e51172e08031b39 0.1s done
#12 exporting manifest list sha256:10f367f79ce82c642d6dcca90ed101522b3b5871a0f5ealb237d1ef905a48 0.0s done
#12 naming to docker.io/library/scaling-demo-backend:latest done
#12 unpacking to docker.io/library/scaling-demo-backend:latest 0.0s done
#12 DONE 0.2s

[+] up 6/6ing provenance for metadata file
✔Image scaling-demo-backend Built 6.5s
✔Network scaling-demo_default Created 0.1s
✔Container scaling-demo-backend-3 Created 0.5s
✔Container scaling-demo-backend-1 Created 0.5s
✔Container scaling-demo-backend-2 Created 0.5s
✔Container frontendcont Created 0.6s
Attaching to frontendcont, backend-1, backend-2, backend-3
backend-3 | * Serving Flask app 'app'
backend-3 | * Debug mode: off
backend-3 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI serv
er instead.
backend-3 | * Running on all addresses (0.0.0.0)

Container scaling-demo-backend-3 Waiting
Container scaling-demo-backend-1 Waiting
Container scaling-demo-backend-2 Waiting
backend-1 | * Serving Flask app 'app'

backend-3 | * Running on http://127.0.0.1:8080
backend-1 | * Debug mode: off
```

```
Command Prompt - docker c

backend-2 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI serv
er instead.
backend-2 | * Running on all addresses (0.0.0.0)
backend-2 | * Running on http://127.0.0.1:8080
backend-2 | * Running on http://172.24.0.4:8080
backend-2 | Press CTRL+C to quit
backend-3 | 127.0.0.1 - - [13/May/2026 13:07:06] "GET / HTTP/1.1" 200 -
backend-1 | 127.0.0.1 - - [13/May/2026 13:07:07] "GET / HTTP/1.1" 200 -
backend-2 | 127.0.0.1 - - [13/May/2026 13:07:08] "GET / HTTP/1.1" 200 -
Container scaling-demo-backend-3 Healthy
Container scaling-demo-backend-1 Healthy
Container scaling-demo-backend-2 Healthy
frontendcont | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
frontendcont | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
frontendcont | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
frontendcont | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
frontendcont | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
frontendcont | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
frontendcont | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
frontendcont | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
frontendcont | /docker-entrypoint.sh: Configuration complete; ready for start up
backend-2 | 172.24.0.5 - - [13/May/2026 13:07:14] "GET / HTTP/1.1" 200 -
frontendcont | 172.24.0.1 - - [13/May/2026:13:07:14 +0000] "GET / HTTP/1.1" 200 70 "-" "Mozilla/5.0 (Windows NT 10.0; W
in64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36"
backend-3 | 127.0.0.1 - - [13/May/2026 13:07:16] "GET / HTTP/1.1" 200 -
backend-1 | 127.0.0.1 - - [13/May/2026 13:07:17] "GET / HTTP/1.1" 200 -
backend-2 | 127.0.0.1 - - [13/May/2026 13:07:18] "GET / HTTP/1.1" 200 -
backend-1 | 172.24.0.5 - - [13/May/2026 13:07:20] "GET / HTTP/1.1" 200 -
frontendcont | 172.24.0.1 - - [13/May/2026:13:07:20 +0000] "GET / HTTP/1.1" 200 70 "-" "Mozilla/5.0 (Windows NT 10.0; W
in64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36"
```

```
Command Prompt - docker c x + v
in64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36"
backend-1 | 172.24.0.5 - - [13/May/2026 13:08:08] "GET / HTTP/1.1" 200 -
backend-1 | 127.0.0.1 - - [13/May/2026 13:08:08] "GET / HTTP/1.1" 200 -
backend-2 | 127.0.0.1 - - [13/May/2026 13:08:09] "GET / HTTP/1.1" 200 -
backend-3 | 172.24.0.5 - - [13/May/2026 13:08:13] "GET / HTTP/1.1" 200 -
frontendcont | 172.24.0.1 - - [13/May/2026:13:08:13 +0000] "GET / HTTP/1.1" 200 70 "-" "Mozilla/5.0 (Windows NT 10.0; W
in64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36"

backend-3 | 127.0.0.1 - - [13/May/2026 13:08:17] "GET / HTTP/1.1" 200 -
backend-1 | 127.0.0.1 - - [13/May/2026 13:08:18] "GET / HTTP/1.1" 200 -

backend-2 | 127.0.0.1 - - [13/May/2026 13:08:19] "GET / HTTP/1.1" 200 -

backend-3 | 127.0.0.1 - - [13/May/2026 13:08:27] "GET / HTTP/1.1" 200 -
backend-1 | 127.0.0.1 - - [13/May/2026 13:08:28] "GET / HTTP/1.1" 200 -

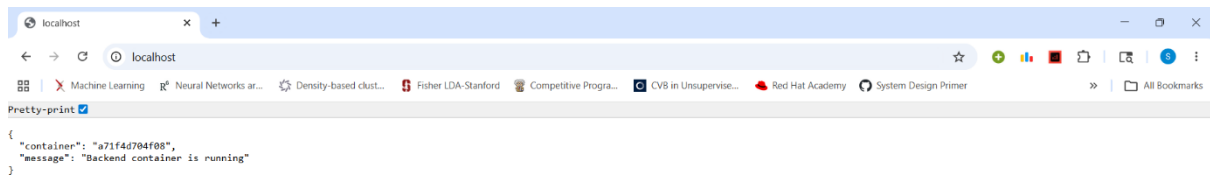
backend-2 | 127.0.0.1 - - [13/May/2026 13:08:29] "GET / HTTP/1.1" 200 -
backend-3 | 127.0.0.1 - - [13/May/2026 13:08:37] "GET / HTTP/1.1" 200 -

backend-1 | 127.0.0.1 - - [13/May/2026 13:08:38] "GET / HTTP/1.1" 200 -
backend-2 | 127.0.0.1 - - [13/May/2026 13:08:39] "GET / HTTP/1.1" 200 -
```

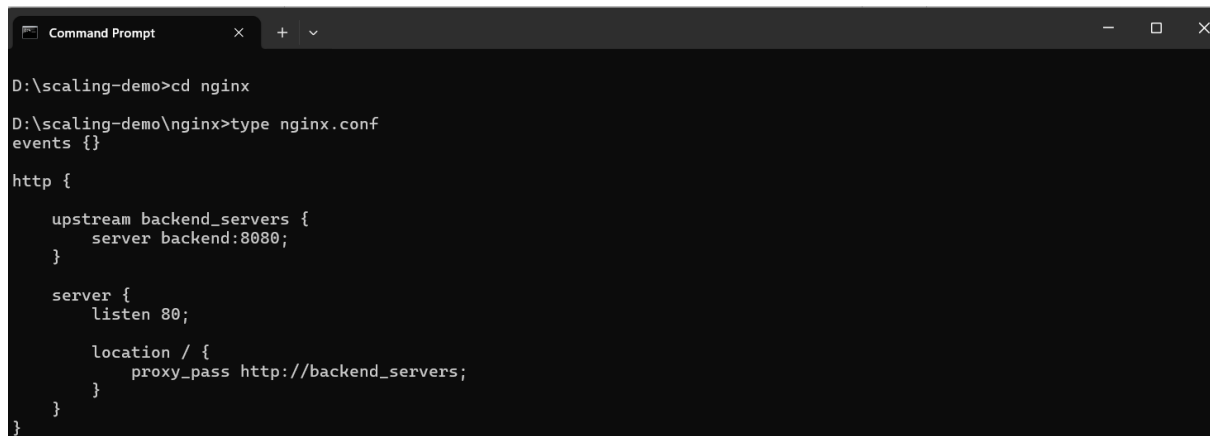
After Scaling:

```
localhost x +
localhost
Machine Learning Neural Networks ar... Density-based clust... Fisher LDA-Stanford Competitive Progra... CVB in Unsupervise... Red Hat Academy System Design Primer
Pretty-print
{
  "container": "5a3b6e2b68bd",
  "message": "Backend container is running"
}
```

```
localhost x +
localhost
Machine Learning Neural Networks ar... Density-based clust... Fisher LDA-Stanford Competitive Progra... CVB in Unsupervise... Red Hat Academy System Design Primer
Pretty-print
{
  "container": "0fb436f4fd6c",
  "message": "Backend container is running"
}
```



```
{
  "container": "a71f4d704f08",
  "message": "Backend container is running"
}
```



```
D:\scaling-demo>cd nginx
D:\scaling-demo\nginx>type nginx.conf
events {}

http {
    upstream backend_servers {
        server backend:8080;
    }

    server {
        listen 80;

        location / {
            proxy_pass http://backend_servers;
        }
    }
}
```

Question 9: Implementing an API Gateway

Real-Life Situation:

A microservices application has multiple backend services, and users should access them through a single entry point.

Task:

Design an API Gateway architecture using an Nginx container as reverse proxy.

Example:

services:

gateway:

image: nginx

userservice:

image: node

orderservice:

image: node

Concepts Covered:

API Gateway pattern, Microservices communication

```
Command Prompt
#18 exporting attestation manifest sha256:f99a23cc93b11df40080eaf2f9d88aa3809e6ca7a67291472fbc96143b9684c5 0.1s done
#18 exporting manifest list sha256:c8d84277950d641a2017181baffca5272c0648fcf18787d8404d754a62654e0f 0.0s done
#18 naming to docker.io/library/api-gateway-userservice:latest
#18 naming to docker.io/library/api-gateway-userservice:latest 0.0s done
#18 unpacking to docker.io/library/api-gateway-userservice:latest 0.0s done
#18 DONE 0.3s

#19 [orderservice] exporting to image
#19 exporting layers done
#19 exporting manifest sha256:c0551984614a7bfca673ecda2c43d10dd1f5da8098526c20f90b4ce5a6560a23 done
#19 exporting config sha256:841847d97eb0ad12bb50b64557f77e138dd2fe4a9c82b84fcb412d92980df281 done
#19 exporting attestation manifest sha256:e344c02029595554633e361a60890ed8afa17b6ef5fc9f2dfa45d366f05aae87 0.1s done
#19 exporting manifest list sha256:e6e6f158c49796b0c58686de4c41a6f5168dc6b9a40df691df8459ab292d238b 0.0s done
#19 naming to docker.io/library/api-gateway-orderservice:latest done
#19 unpacking to docker.io/library/api-gateway-orderservice:latest 0.0s done
#19 DONE 0.3s

#20 [userservice] resolving provenance for metadata file
#20 DONE 0.0s

#21 [orderservice] resolving provenance for metadata file
#21 DONE 0.0s
[+] up 6/6
✔Image api-gateway-userservice Built 6.3s
✔Image api-gateway-orderservice Built 6.3s
✔Network api-gateway_default Created 0.1s
✔Container orderservice Created 0.3s
✔Container userservice Created 0.3s
✔Container gateway Created 0.3s
```

```
localhost/users/ localhost/orders/ localhost:3001 localhost:3002
localhost/users/
{
  "service": "User Service",
  "message": "User service is running"
}
```

```
localhost/users/ localhost/orders/ localhost:3001 localhost:3002
localhost/orders/
{
  "service": "Order Service",
  "message": "Order service is running"
}
```

```
localhost/users/ x localhost/orders/ x localhost:3001 x localhost:3002 x +
localhost:3001
Machine Learning Neural Networks ar... Density-based clust... Fisher LDA-Stanford Competitive Progra... CVB in Unsupervise... Red Hat Academy System Design Primer
Pretty-print
{
  "service": "User Service",
  "message": "User service is running"
}
```

```
localhost/users/ x localhost/orders/ x localhost:3001 x localhost:3002 x +
localhost:3002
Machine Learning Neural Networks ar... Density-based clust... Fisher LDA-Stanford Competitive Progra... CVB in Unsupervise... Red Hat Academy System Design Primer
Pretty-print
{
  "service": "Order Service",
  "message": "Order service is running"
}
```

```
Command Prompt
Folder PATH listing for volume New Volume
Volume serial number is 000000DF AEE6:4A19
D:.
├── docker-compose.yml
├── gateway
│   └── nginx.conf
├── orderservice
│   ├── dockerfile
│   ├── package.json
│   └── server.js
└── userservice
    ├── dockerfile
    ├── package.json
    └── server.js

D:\api-gateway>type docker-compose.yml
services:
  orderservice:
    build: ./orderservice
    container_name: orderservice
    ports:
      - "3002:3002"
    restart: unless-stopped

  userservice:
    build: ./userservice
    container_name: userservice
    ports:
      - "3001:3001"
    restart: unless-stopped

  gateway:
    image: nginx:latest
    container_name: gateway
    ports:
      - "80:80"
    restart: unless-stopped
    volumes:
      - ./gateway/nginx.conf:/etc/nginx/nginx.conf
    depends_on:
      - orderservice
      - userservice

D:\api-gateway>
```

```
Command Prompt
D:\api-gateway\orderservice>type dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3002
CMD ["node", "server.js"]
D:\api-gateway\orderservice>cd ..

D:\api-gateway>cd userservice

D:\api-gateway\userservice>type dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3001
CMD ["node", "server.js"]
D:\api-gateway\userservice>cd ..

D:\api-gateway>cd gatwat
The system cannot find the path specified.

D:\api-gateway>cd gateway

D:\api-gateway\gateway>type nginx.conf
events {}

http {

    server {

        listen 80;

        location /users/ {
            proxy_pass http://userservice:3001;
        }

        location /orders/ {
            proxy_pass http://orderservice:3002;
        }

    }
}
D:\api-gateway\gateway>
```

Docker Compose Based Multi-Container Web Application with Redis Persistence

```
Command Prompt
D:\redis-devops>type docker-compose.yml
services:
  frontend:
    build: ./frontend
    container_name: frontend
    ports:
      - "3000:80"
    restart: unless-stopped
    depends_on:
      - backend

  backend:
    build: ./backend
    container_name: backend
    ports:
      - "5000:5000"
    restart: unless-stopped
    environment:
      REDIS_HOST: redis
      REDIS_PORT: 6379
    depends_on:
      - redis

  redis:
    image: redis:7-alpine
    container_name: redis
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
    restart: unless-stopped
    command: redis-server --appendonly yes

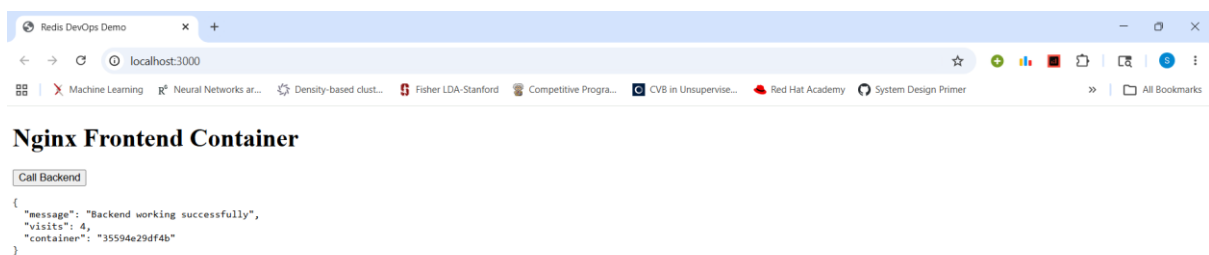
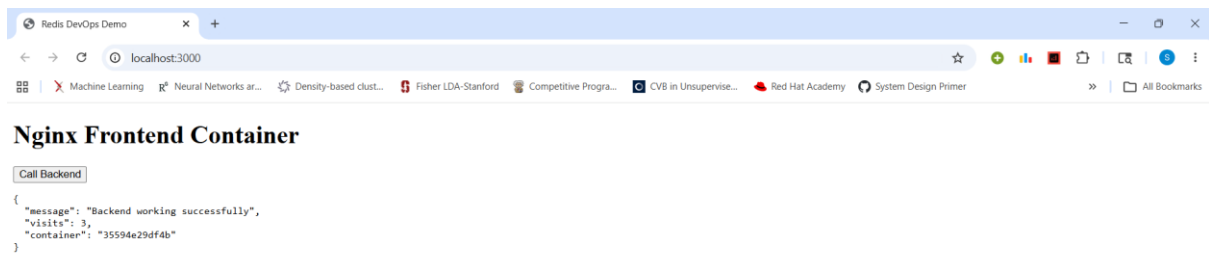
volumes:
  redis-data:
```

```
D:\redis-devops>tree /F
Folder PATH listing for volume New Volume
Volume serial number is 00000077 AEE6:4A19
D: .
|   docker-compose.yml
|   |
|   |-- backend
|   |   |   dockerfile
|   |   |   package.json
|   |   |   server.js
|   |   |
|   |-- frontend
|   |   |   dockerfile
|   |   |   nginx.conf
|   |   |
|   |-- public
|   |       index.html
```

```
D:\redis-devops>type backend\dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 5000
CMD ["npm", "start"]
D:\redis-devops>type frontend\dockerfile
FROM nginx:latest
COPY public /usr/share/nginx/html
COPY nginx.conf /etc/nginx/nginx.conf
EXPOSE 80
D:\redis-devops>
```

```
Command Prompt - docker c x + v
#20 exporting manifest sha256:21161549bd27dcc8684fe66e7134799aaflaalfda78899949f2e137ace80b10a9 done
#20 exporting config sha256:86a29885dcb883ce063bb17f7135ba381cd7bb714f14063b00275c32e8d81afe done
#20 exporting attestation manifest sha256:45319f044c364f0e4ef0aea8ab9d1931d405b391637e737333e132217dd29f7 0.0s done
#20 exporting manifest list sha256:4d7d46694a3edac0e0f537ee96656b84464ea16252c3c7d1becfa5d5c63332 0.0s done
#20 naming to docker.io/library/redis-devops-backend:latest
#20 naming to docker.io/library/redis-devops-backend:latest done
#20 unpacking to docker.io/library/redis-devops-backend:latest 0.0s done
#20 DONE 0.2s

#21 [backend] resolving provenance for metadata file
#21 DONE 0.0s
[+] up 7/7
✓Image redis-devops-frontend Built 2.4s
✓Image redis-devops-backend Built 2.4s
✓Network redis-devops-default Created 0.1s
✓Volume redis-devops-redis-data Created 0.0s
✓Container redis Created 0.2s
✓Container backend Created 0.1s
✓Container frontend Created 0.2s
Attaching to backend, frontend, redis
redis 1:C 14 May 2026 11:23:04.232 * o000o000o000o Redis is starting o000o000o000o
redis 1:C 14 May 2026 11:23:04.232 * Redis version=7.4.9, bits=64, commit=00000000, modified=0, pid=1, just started
redis 1:C 14 May 2026 11:23:04.232 * Configuration loaded
redis 1:M 14 May 2026 11:23:04.233 * monotonic clock: POSIX clock_gettime
redis 1:M 14 May 2026 11:23:04.235 * Running mode=standalone, port=6379.
redis 1:M 14 May 2026 11:23:04.236 * Server initialized
redis 1:M 14 May 2026 11:23:04.241 * Creating AOF base file appendonly.aof.1.base.rdb on server start
redis 1:M 14 May 2026 11:23:04.248 * Creating AOF incr file appendonly.aof.1.incr.aof on server start
redis 1:M 14 May 2026 11:23:04.248 * Ready to accept connections tcp
backend > backend@1.0.0 start
backend > node server.js
frontend /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
frontend /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
frontend /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
frontend 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
frontend 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
frontend /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
frontend /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
frontend /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
frontend /docker-entrypoint.sh: Configuration complete; ready for start up
backend Connected to Redis
backend Backend running on port 5000
```



The above two images are before container stop

```
D:\redis-devops>docker compose down
[+] down 4/4
✓Container frontend      Removed      0.1s
✓Container backend      Removed      0.1s
✓Container redis         Removed      0.1s
✓Network redis-devops_default Removed      0.3s
D:\redis-devops>
```

Container stop

```
D:\redis-devops>docker compose up
[+] up 4/4
✓Network redis-devops_default Created      0.0s
✓Container redis           Created      0.2s
✓Container backend         Created      0.2s
✓Container frontend        Created      0.2s
Attaching to backend, frontend, redis
redis 1:C 14 May 2026 11:26:45.769 * o000o000o000o Redis is starting o000o000o000o
redis 1:C 14 May 2026 11:26:45.769 * Redis version=7.4.9, bits=64, commit=00000000, modified=0, pid=1, just started
redis 1:C 14 May 2026 11:26:45.769 * Configuration loaded
redis 1:M 14 May 2026 11:26:45.771 * monotonic clock: POSIX clock_gettime
redis 1:M 14 May 2026 11:26:45.776 * Running mode=standalone, port=6379.
redis 1:M 14 May 2026 11:26:45.778 * Server initialized
redis 1:M 14 May 2026 11:26:45.779 * Reading RDB base file on AOF loading...
redis 1:M 14 May 2026 11:26:45.779 * Loading RDB produced by version 7.4.9
redis 1:M 14 May 2026 11:26:45.779 * RDB age 221 seconds
redis 1:M 14 May 2026 11:26:45.779 * RDB memory usage when created 0.90 Mb
redis 1:M 14 May 2026 11:26:45.779 * RDB is base AOF
redis 1:M 14 May 2026 11:26:45.779 * Done loading RDB, keys loaded: 0, keys expired: 0
redis 1:M 14 May 2026 11:26:45.780 * DB loaded from base file appendonly.aof.1.base.rdb: 0.001 seconds
redis 1:M 14 May 2026 11:26:45.781 * DB loaded from incr file appendonly.aof.1.incr.aof: 0.001 seconds
redis 1:M 14 May 2026 11:26:45.781 * DB loaded from append only file: 0.003 seconds
redis 1:M 14 May 2026 11:26:45.781 * Opening AOF incr file appendonly.aof.1.incr.aof on server start
redis 1:M 14 May 2026 11:26:45.781 * Ready to accept connections tcp
backend | > backend@1.0.0 start
backend | > node server.js
frontend | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
frontend | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
frontend | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
frontend | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
frontend | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
frontend | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
frontend | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
frontend | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
frontend | /docker-entrypoint.sh: Configuration complete; ready for start up
backend | Connected to Redis
backend | Backend running on port 5000
```



Nginx Frontend Container

```
Call Backend

{
  "message": "Backend working successfully",
  "visits": 5,
  "container": "7e0011b663c7"
}
```

Volume persistence in action thanks to docker volumes and startup command --appendonly yes